SAICC XVI · MINICURSO

Introdução ao Desenvolvimento Mobile com Flutter

Do Dart ao primeiro app cross-platform: widgets, estado, tema e layout construindo um app de lista de tarefas

Matheus Silveira

Apresentação

- Matheus Silveira, graduando em Engenharia da Computação
- Principais stacks que trabalho: AI/ML & desenvolvimento mobile com Flutter
- Hoje: **~2 horas** para sair do zero e chegar a um app mvp

Teoria

~35 min de slides: Dart, Flutter, layout e estado

Prática

~80 min construindo o app de tarefas juntos

Fechamento

5 min para próximos passos e perguntas

O que vamos construir

- App de **tarefas** (TODO list)
- Adicionar, concluir, remover e desfazer
- Tema **Material 3**, claro e escuro
- Dados salvos no aparelho (se der tempo)

Combinados

- Perguntas a qualquer momento
- Travou? Levante a mão, sem esperar
- Digite o código, não cole
- Cada etapa tem um checkpoint no repositório

DART

A linguagem do Flutter

- Criada pelo Google, **fortemente tipada** e orientada a objetos
- Sintaxe familiar: Java, C#, JavaScript, Kotlin
- **Null safety** embutida na linguagem
- `async` / `await` nativo

DART

Variáveis e null safety

```dart
var name = 'Ana';                  // variáveis declaradas com o modificador var tem seu tipo inferido em
final age = 21;                    // variáveis declaradas com o modificador final só podem ser atribuídas
const pi = 3.14159;                // variáveis declaradas com o modificador const são constantes em tempo

String? nickname;                  // Nesse caso, o "?" faz com que essa váriavel possa assumir valores nu
print(nickname?.length);           // acesso seguro: null se for null
final shown = nickname ?? name; // a variável shown recebe "name" caso nickname seja null
print(nickname!.length);           // "!" diz ao compilador que nickname nunca pode ser nulo (resulta em e
```

Sem `?`, a variável **nunca** é nula e o compilador garante isso.

DART

Funções e classes

```dart
class Todo {
  const Todo({required this.title, this.isDone = false}); // esse é o construtor da classe. o modificad
                                                          // não possui um valor padrão, diferentemente

  final String title; // definimos o tipo dos atributos do construtor nesse padrão; Sempre com final.
  final bool isDone;

  Todo copyWith({bool? isDone}) =>    // aqui é usado uma arrow function para criar outra instância de
      Todo(title: title, isDone: isDone ?? this.isDone);           // esse estilo de código é comum em
}

final todo = Todo(title: 'Estudar Dart');
final done = todo.copyWith(isDone: true);
```

DART

Coleções

```dart
final names = ['Ana', 'Bia', 'Caio'];          // List<String>
final ages = {'Ana': 21, 'Bia': 22};             // Map<String, int>    é como se fosse o dict do pytho

// spread, collection if e collection for
final all = [...names, 'Duda']; // aqui os "..." antes de names indica um desempacotamento
final list = [
  for (final n in names) if (n != 'Bia') n,
];
```

O `for` e o `if` dentro de listas aparecem muito na construção de telas.

DART

Condicionais: if / else

```dart
final age = 17;

if (age >= 18) {
  print('maior de idade');
} else if (age >= 16) {
  print('voto facultativo');
} else {
  print('menor de idade');
}

if (isLoggedIn) print('Bem-vindo!'); // sem chaves
```

Operadores de comparação e lógica

`==` `!=` `>` `<` `>=` `<=`

`&&` (e) `||` (ou) `!` (não)

- A condição precisa ser **bool**: não existe "truthy" como no JavaScript
- `if (list)` ou `if (0)` não compilam
- Use `list.isEmpty`, `n == 0`, `text != null`

DART

Operador ternário e operadores nulos

condição ? **se verdadeiro** : **se falso**

```dart
final status = isDone ? 'Concluída' : 'Pendente';    // ternário

final label = nickname ?? 'sem apelido';             // ?? usa o padrão se for null
nickname ??= 'Visitante';                            // ??= atribui só se for null
final size = nickname?.length;                       // ?. devolve null se for null

final tag = n == 0 ? 'vazio' : n == 1 ? 'um' : 'vários'; // aninhado: evite
```

O ternário é uma **expressão** (devolve valor); o `if` é um **comando**. Por isso o ternário cabe dentro de parâmetros de widgets.

DART

enum e switch

```dart
enum Priority { low, medium, high }

// switch como expressão (Dart 3): devolve valor e exige cobrir todos os casos
final label = switch (priority) {
  Priority.high => 'Alta',
  Priority.medium => 'Média',
  Priority.low => 'Baixa',
};

// switch clássico: não precisa de break
switch (day) {
  case 6 || 7:
    print('fim de semana');
  default:
    print('dia útil');
}
```

Com `enum` o compilador avisa se você esquecer um caso. Ótimo para estados da tela.

DART

Loops: for, while e companhia

```dart
for (var i = 0; i < 3; i++) {            // for clássico
  print(i);
}
for (final name in names) {              // for-in: o mais usado
  print(name);
}
names.forEach(print);                    // forEach

while (n > 0) { n--; }                   // while
do { n++; } while (n < 3);               // do-while

for (final x in numbers) {
  if (x.isOdd) continue;                 // pula para a próxima volta
  if (x > 8) break;                      // sai do loop
  print(x);
}
```

DART

Métodos que substituem loops

```dart
final total = prices.fold(0.0, (sum, p) => sum + p);   // acumula um valor
final anyDone = todos.any((t) => t.isDone);             // algum cumpre?
final allDone = todos.every((t) => t.isDone);           // todos cumprem?
final pending = todos.where((t) => !t.isDone).length;   // filtra e conta
final first = todos.firstWhere((t) => t.id == id);      // erro se não achar
final maybe = todos.where((t) => t.id == id).firstOrNull; // null se não achar

final sorted = [...todos]..sort((a, b) => a.title.compareTo(b.title));
final evens = List.generate(5, (i) => i * 2);           // [0, 2, 4, 6, 8]
```

Menos variáveis temporárias e menos bugs: diga **o que** quer, não **como** percorrer.

DART

Funções: três tipos de parâmetro

```dart
int sum(int a, int b) => a + b;                          // posicionais + seta

String greet(String name, [String hello = 'Olá']) =>     // opcional posicional
    '$hello, $name!';

void show({required String title, int lines = 1, Color? color}) {} // nomeados

sum(1, 2);
greet('Ana');
show(title: 'Oi', color: Colors.red);                    // ordem livre

final twice = (int x) => x * 2;                          // função anônima
```

Widgets usam **nomeados**; callbacks como `onPressed: () { ... }` são funções anônimas.

DART

Mais detalhes úteis do Dart

```dart
print('Olá, $name! Você tem ${todos.length * 2} itens.');  // interpolação

if (value is String) {                  // "is" testa o tipo...
  print(value.length);                  // ...e já promove para String
}
final text = value as String;           // "as" converte (erro se não for)

final list = <int>[]..add(1)..add(2)..add(3);  // cascata: vários usos do mesmo objeto

late final String title;                // "late": será definida antes do primeiro uso
const items = ['a', 'b'];               // const: imutável, criada em tempo de compilação

final (a, b) = (1, 'dois');             // records e desestruturação (Dart 3)
```

DART

Assíncrono: Future, async e await

```dart
Future<String> loadName() async {
  await Future.delayed(const Duration(seconds: 1)); // simula disco/rede
  return 'Gabriel';
}

Future<void> main() async {
  print('carregando...');
  final name = await loadName(); // espera sem travar a interface
  print('Olá, $name!');
}
```

Disco, rede e banco de dados são **assíncronos**.

Usaremos `await` na persistência.

FLUTTER

O que é o Flutter

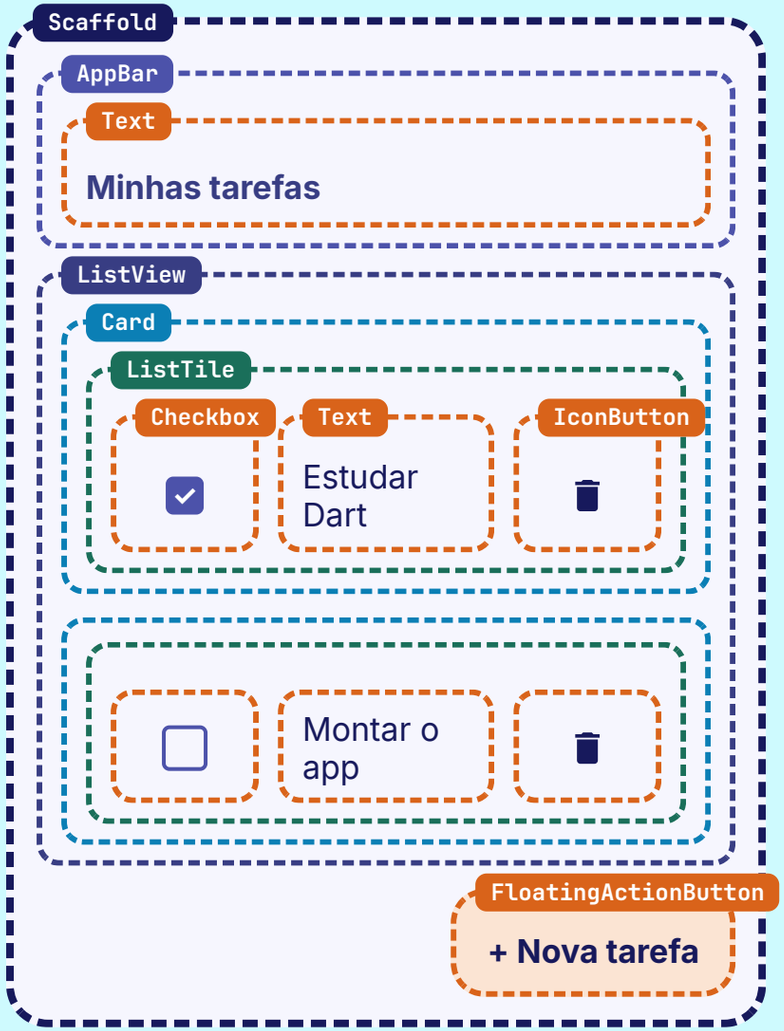
- Framework **open source** do Google para interfaces
- **Um código**: Android, iOS, web, desktop
- Interface **declarativa**: você descreve como a tela deve ser para um estado

Por que escolher

- Produtividade com hot reload
- Desempenho próximo ao nativo
- Catálogo grande de widgets prontos
- Comunidade e pacotes em **pub.dev**

FLUTTER

Tudo é widget: cada caixa é um widget




```
Scaffold
 ├─ AppBar
 │   └─ Text
 ├─ ListView
 │   └─ Card
 │       └─ ListTile
 │           ├─ Checkbox
 │           ├─ Text
 │           └─ IconButton
 └─ FloatingActionButton
```

- A interface é uma **árvore de widgets**
- Widgets pequenos, **compostos** entre si
- Padding, alinhamento e tema também são widgets

FLUTTER

StatelessWidget e StatefulWidget

StatelessWidget

Não lembra de nada. A tela depende só do que recebe.

```dart
class Title extends StatelessWidget {
  const Title(this.text, {super.key});
  final String text;

  @override
  Widget build(BuildContext context) {
    return Text(text);
  }
}
```

StatefulWidget

Tem um objeto `State` que sobrevive às reconstruções.

```dart
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;
  // build(): usa 'count'
}
```

Pergunta-guia: "esse widget precisa lembrar de algo?"

FLUTTER

Ciclo de vida e hot reload

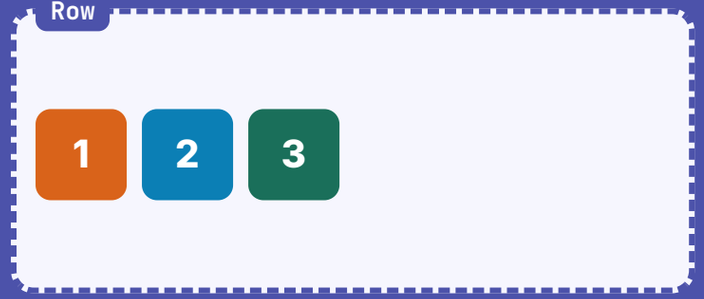


- **Hot reload** (`r`): injeta o código novo e mantém o estado
- **Hot restart** (`R`): recomeça o app do zero
- `build` roda muitas vezes: deve ser **rápido e sem efeitos colaterais**
- Use `const`: o Flutter pula a reconstrução desses widgets

LAYOUT

Row, Column e Stack

Row



Filhos na **horizontal**

Eixo principal: →

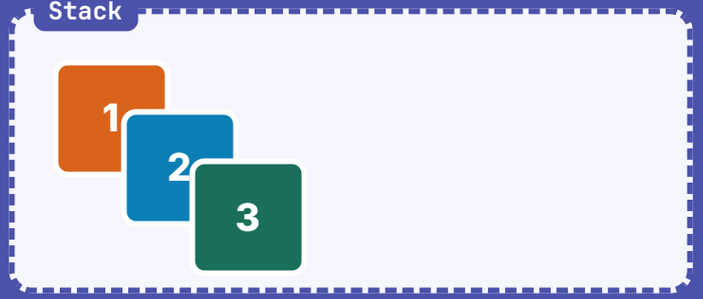
Column



Filhos na **vertical**

Eixo principal: ↓

Stack



Filhos **sobrepostos**

O último fica por cima

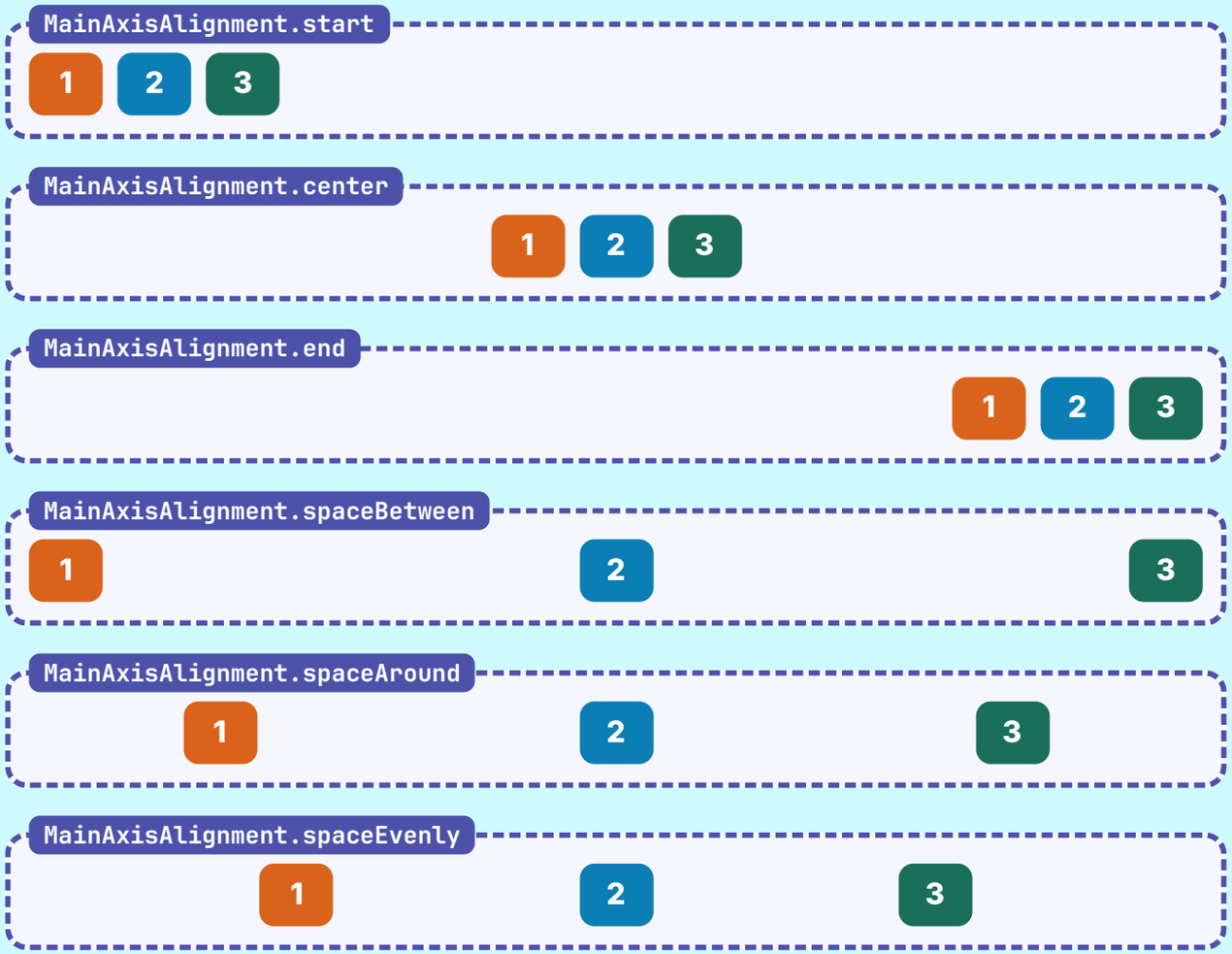
Nos próximos slides: `mainAxisAlignment` e `crossAxisAlignment` em ação.

LAYOUT

Row: `mainAxisAlignment`

```
Row(
  mainAxisAlignment:
      MainAxisAlignment.spaceBetween,
  crossAxisAlignment:
      CrossAxisAlignment.center,
  children: [
    Box('1'),
    Box('2'),
    Box('3'),
  ],
)
```

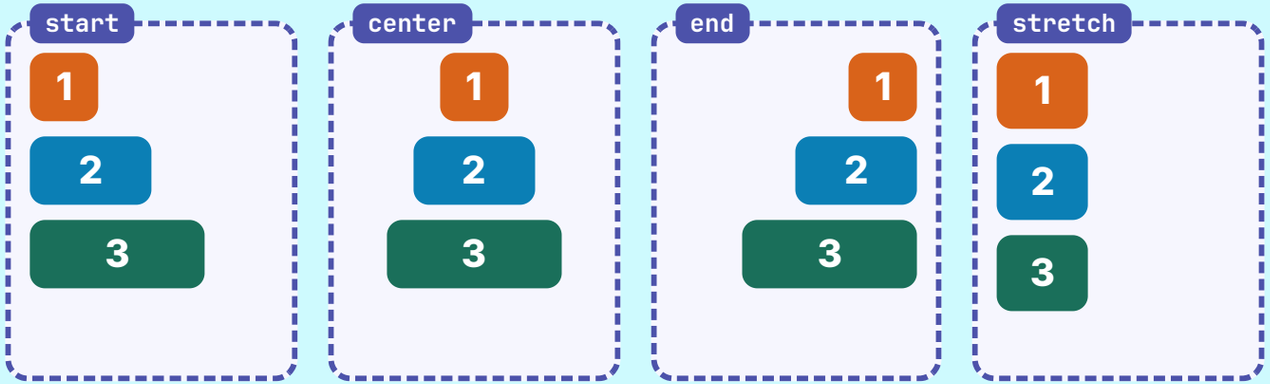
- **Eixo principal** da `Row`: horizontal
- **Eixo cruzado**: vertical (`crossAxisAlignment`)
- Sem espaço sobrando, o `mainAxisAlignment` não tem efeito



LAYOUT

Column: `crossAxisAlignment`

```
Column(
  crossAxisAlignment:
      CrossAxisAlignment.start,
  children: [
    Box('1'),
    Box('2'),
    Box('3'),
  ],
)
```



- Na `Column` o eixo principal é **vertical**
- O eixo cruzado é **horizontal**: alinha cada filho na largura
- `stretch` força os filhos a ocuparem toda a largura
- `mainAxisSize: MainAxisSize.min` encolhe a coluna ao conteúdo

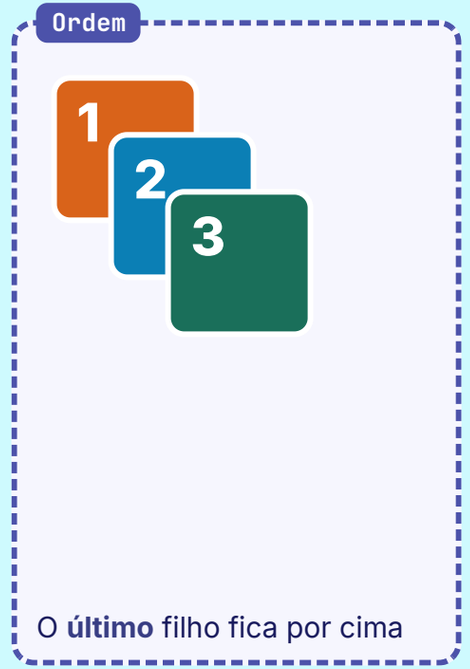
Valores de `CrossAxisAlignment`. Na `Row` é o mesmo, na vertical.

LAYOUT

Stack: sobrepondo widgets

```
Stack(
  children: [
    Container(color: Colors.blue),
    Positioned(
      top: 12, right: 12,
      child: Badge(),
    ),
    Align(
      alignment: Alignment.center,
      child: PlayIcon(),
    ),
    Positioned(
      left: 0, right: 0, bottom: 0,
      child: TitleBar(),
    ),
  ],
)
```



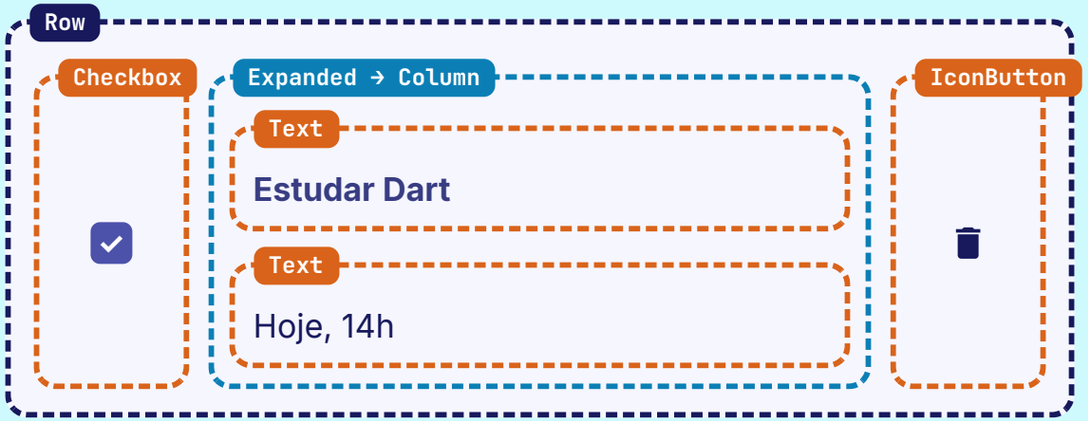


`Positioned` usa distâncias das bordas; `Align` usa posição relativa (de -1 a 1).

LAYOUT

Juntando tudo: Row + Column + Expanded

```
Row(
  children: [
    Checkbox(value: isDone),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(title),
          Text('Hoje, 14h'),
        ],
      ),
    ),
    IconButton(
      icon: Icon(Icons.delete),
      onPressed: onDelete,
    ),
  ],
)
```



O `Expanded` faz a coluna ocupar **todo o espaço que sobra** entre o checkbox e o botão.

LAYOUT

Dando forma ao layout

- `Padding` e `SizedBox`: espaços
- `Center` e `Align`: posicionamento
- `Container`: cor, borda e tamanho

LAYOUT

Listas que rolam

```
ListView.separated(
  itemCount: todos.length,
  separatorBuilder: (_, _) => const SizedBox(height: 8),
  itemBuilder: (context, index) => TodoTile(todo: todos[index]),
)
```

- `.builder` cria só os itens **visíveis** (lista "preguiçosa")
- `GridView` para grades, `SingleChildScrollView` para conteúdo curto
- Não coloque `ListView` direto dentro de `Column` sem `Expanded`

LAYOUT

Condicionais na interface

```dart
@override
Widget build(BuildContext context) {
  // 1. retorno antecipado
  if (_loading) {
    return const CircularProgressIndicator();
  }
  return Column(
    children: [
      // 2. ternário no valor
      Text(todos.isEmpty
          ? 'Nada por fazer'
          : '${todos.length} tarefas'),
      // 3. if dentro dos filhos
      if (todos.isEmpty) const EmptyState(),
      if (hasError) const Banner() else const Divider(),
    ],
  );
}
```

Ternário em propriedades

```dart
color: isDone ? Colors.grey : primary,

decoration: isDone
    ? TextDecoration.lineThrough
    : null,

child: _loading
    ? const Spinner()
    : const TodoList(),
```

- O `if` dentro de `children` **não usa chaves** nem `return`
- Para "não mostrar nada": `SizedBox.shrink()`

LAYOUT

Loops na interface

```
Column(
  children: [
    // for dentro da lista
    for (final todo in todos)
      TodoTile(todo: todo),
    // for clássico
    for (var i = 0; i < 3; i++)
      Text('Item $i'),
    // map + spread
    ...todos.map((t) => Text(t.title)),
    // if + spread de vários widgets
    if (todos.isNotEmpty) ...[
      const Divider(),
      const Text('Fim da lista'),
    ],
  ],
)
```

Qual usar?

- **for / map**: poucos itens, até algumas dezenas
- **ListView.builder**: listas grandes ou de tamanho desconhecido

Cuidado

`forEach` não devolve widgets: não serve dentro de `children`.

Use `for` ou `map`.

ESTADO

O que é estado

UI = f(estado)

- Estado é qualquer dado que **muda** e afeta a tela
- No app: a lista de tarefas e se estamos carregando
- Mudou o estado, o Flutter reconstrói a interface

Estado local (efêmero)

Vale para **um** widget ou tela. Ex.: texto digitado, aba atual.

Estado compartilhado

Várias telas usam. Ex.: usuário logado, carrinho.

ESTADO

setState

Evento (toque) → setState → build roda de novo → Tela atualiza

```
void _toggle(Todo todo) {
  setState(() {
    _todos = [
      for (final t in _todos)
        if (t.id == todo.id) t.copyWith(isDone: !t.isDone) else t,
    ];
  });
}
```

Alterar a variável **sem** `setState` não atualiza a tela.

ESTADO

Imutabilidade e levantar o estado

Dados imutáveis

- Campos `final` e construtor `const`
- `copyWith` em vez de editar o objeto
- Nova lista em vez de `list.add`

Levantar o estado

- Quem **mostra** o dado recebe valores
- Quem **muda** o dado recebe callbacks
- `TodoTile(todo, onToggle, onDelete)`

Dados descem, eventos sobem.

BOAS PRÁTICAS

Código limpo em Flutter

- Widgets **pequenos**, um propósito cada
- `const` sempre que possível
- Nomes claros; privado com `_`
- Pastas por responsabilidade

```
lib/
  data/        persistência
  models/      Todo
  screens/     telas
  theme/       cores e temas
  widgets/     componentes
```

Evite cores e textos fixos no widget. Use `Theme.of(context)`.

ESTADO

E quando o app crescer?

`setState` resolve estado de **uma tela**. Com várias telas compartilhando dados, outras ferramentas ajudam:

Provider
Simples e oficial no início

Riverpod
Evolução moderna e testável

Bloc / Cubit
Eventos e estados explícitos

Hoje vamos só de `setState`: é a base de todas elas.

TEMA

Material Theme Builder

1. Abra o **Material Theme Builder**
2. Escolha a cor primária
3. Veja o preview claro e escuro
4. **Export** → Flutter (Dart)
5. Copie os `ColorScheme` para `lib/theme/color_scheme.dart`

material-foundation.github.io

/material-theme-builder

Demonstração ao vivo agora.

VS CODE

IntelliSense: `Ctrl` + `Espaço`

- Abre a lista de **sugestões** no ponto do cursor
- Mostra parâmetros, métodos e propriedades (experimente após o `.`)
- `Tab` ou `Enter` aceita a sugestão; `Esc` fecha
- `Ctrl` + `Shift` + `Espaço`: dicas dos parâmetros da função
- Passe o mouse sobre um nome para ver a **documentação**

Dica: digite só as iniciais. `cAA` encontra `crossAxisAlignment`.

```
Text(
  'Olá, SAICC!',
  st
)
```

style	TextStyle?
strutStyle	StrutStyle?
softWrap	bool?
semanticsLabel	String?

VS CODE

Formatando o código

`Ctrl` + `A`

seleciona tudo



`Ctrl` + `Shift` + `I`

formata

Antes

```
Column(children: [Text('a'),
 Text('b')
,],)
```

Depois

```
Column(
  children: [
    Text('a'),
    Text('b'),
  ],
)
```

- No **Windows**, o atalho é `Shift` + `Alt` + `F` (no macOS, `Shift` + `Option` + `F`)
- A **vírgula final** (trailing comma) faz o formatador quebrar um item por linha
- Ative `editor.formatOnSave` para formatar sempre que salvar

VS CODE

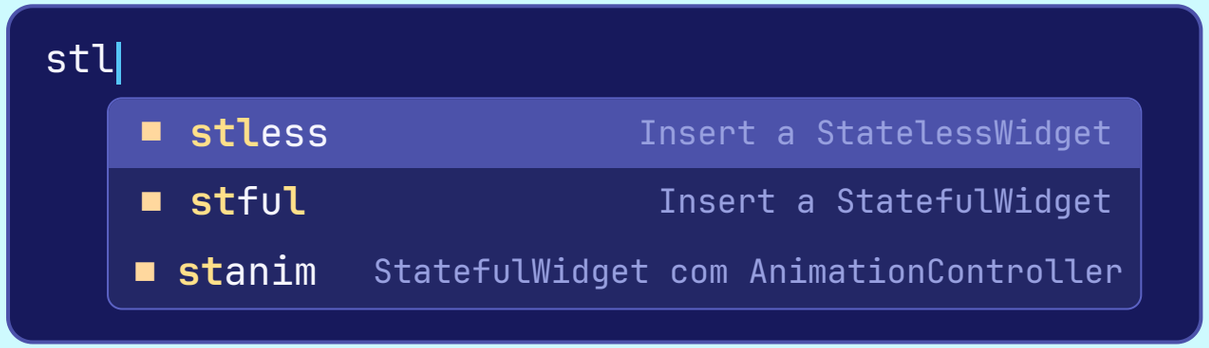
Atalhos que você vai usar o tempo todo

`Ctrl` + `.`	**Quick Fix**: envolver com `Padding`, `Column`, `Center`; extrair widget
`Ctrl` + `P`	Abrir arquivo pelo nome
`F12` ou `Ctrl`+clique	Ir para a **definição** do widget
`Ctrl` + `/`	Comentar ou descomentar a linha
`Ctrl` + `S`	Salva e dispara o **hot reload**
`Ctrl` + `Shift` + `P`	Paleta de comandos (`Flutter: New Project`)
`F2`	Renomear em todo o projeto
`Alt` + `↑` / `↓`	Mover a linha para cima ou para baixo

No macOS, troque `Ctrl` por `Cmd`.

VS CODE

Snippets do Flutter: `stless`



- Digite `stl` e use o IntelliSense (**Ctrl** + **Espaço**)
- Escolha `stless` e aperte **Tab**
- Digite o **nome da classe**: ele troca nos dois lugares

Resultado

```dart
class MeuWidget extends StatelessWidget {
  const MeuWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

`Placeholder` é só um espaço reservado: troque pelo seu widget.

VS CODE

Snippets do Flutter: `stful`

```dart
class Contador extends StatefulWidget {
  const Contador({super.key});

  @override
  State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  @override
  Widget build(BuildContext context) {
    return const Placeholder();
  }
}
```

- Digite `stful` + **Tab**: o snippet cria **duas classes**
- O nome é preenchido nos três pontos de uma vez
- O estado e o `setState` ficam na classe `_ContadorState`

Qual escolher?

`stless`: a tela só depende do que recebe

`stful`: o widget precisa lembrar de algo

VS CODE

Outros snippets úteis

Flutter

`stless`	StatelessWidget
`stful`	StatefulWidget com State
`stanim`	StatefulWidget com AnimationController

Dart

`main`	função `main`
`if / ife`	if e if/else
`for / fori`	for e for-in
`switch`	switch
`try`	try/catch
`class`	definição de classe

Digite o prefixo, aperte **Ctrl** + **Espaço** e escolha o item com o ícone de snippet. Os snippets são da extensão **Flutter** (Dart Code), que você já instalou.

HANDS-ON

Roteiro: 6 checkpoints

Checkpoint 1	Criar o projeto e o `main.dart`	`01-projeto-base`
Checkpoint 2	Aplicar o tema do Theme Builder	`02-tema`
Checkpoint 3	Modelo `Todo`, `TodoTile` e lista fixa	`03-lista-fixa`
Checkpoint 4	`setState`, bottom sheet e formulário	`04-adicionar-tarefas`
Checkpoint 5	Concluir, remover e desfazer	`05-concluir-e-remover`
Checkpoint 6	Salvar com `shared_preferences` (se der tempo)	`06-persistencia`

Cada pasta de `checkpoints/` tem o que o seu `lib/` deve ter ao final da etapa. Travou? Copie o checkpoint para o seu `lib/` e siga com a turma.

PRÓXIMOS PASSOS

Para continuar aprendendo

- Navegação entre telas: `Navigator` e `go_router`
- Estado compartilhado: Provider, Riverpod, Bloc
- Banco local (SQLite, Drift) e APIs REST (`http`, `dio`)
- Testes de widget e de integração
- Animações e publicação nas lojas

Referências

- docs.flutter.dev
- dart.dev/
- pub.dev
- Widget of the Week (YouTube)

OBRIGADO!

Perguntas?

Obrigado pela sua presença!



linkedin.com/in/matheuspsilveira